Sensor, odometry and navigation integration

Developer guide V0.1 | Source software v1.1.0 | 2026-10-11

Software integration examples. Measure transforms, footprint and mounting geometry on the delivered configuration.

Status: Software reference

Assign each coordinate transform one owner

The driver provides wheel odometry and, by default, odom -> base_footprint. The robot description connects the chassis links; a sensor needs its own measured transform to the mounting link.

Connection	Owner
odom -> base_footprint	Driver, or the chosen localisation estimator
base_footprint -> base_link and chassis links	Robot description / robot_state_publisher
Chassis mounting link -> sensor link	Your measured payload transform

If robot_localization or another estimator publishes the odometry transform, disable the driver copy using the launch publish_tf argument. Two publishers for the same transform produce an inconsistent tree.

```
ros2 launch luchs_bringup luchs_a.launch.py \
  build:=4ws dry_run:=true publish_tf:=false
```

Describe a sensor mount

The A model includes rail_left_link and rail_right_link. The source describes two deck rails 200 mm apart. Use the actual mount and measured lever arm; do not copy an example offset as a mechanical dimension.

```
<joint name="sensor_mount" type="fixed">
  <parent link="rail_left_link"/>
  <child link="sensor_link"/>
  <origin xyz="0.25 -0.10 0.08" rpy="0 0 0"/>
</joint>
<link name="sensor_link"/>
```

The numbers above are an illustrative source offset that must be replaced with measured values. Check mounting constraints and payload power in the approved hardware documentation before installation. B mounting details and models remain preliminary.

Fuse wheel odometry with external sensors

- Configure your estimator with the actual odometry and sensor topic names.

- Use a planar configuration for a ground vehicle and enable only measured channels.

- For skid-steer and Ackermann models, do not treat lateral velocity as an independent measurement.

- Draft LCI imu/data_raw supplies angular rate and acceleration without orientation; use it only if fitted and verified.

- An external GNSS/INS may accept wheel-speed input. Match units and measured lever arms to that sensor configuration.

The public integration source contains robot_localization and GNSS/INS examples. They are configuration examples, not calibration results for your sensor or vehicle.

Prepare navigation geometry

1. Measure the delivered robot footprint including the payload and protrusions.

2. Choose a motion model matching the drivetrain: lateral commands for supported A-4WS, differential for skid-steer, or Ackermann for steering-axle builds.

3. Set wheelbase, wheel radius and turning constraints from the confirmed unit configuration.

4. Inspect the transform tree, odometry direction and sensor timing in simulation before a controlled hardware trial.

Scope note. The B public software source contains unresolved geometry and steering-range differences. Its envelope model and defaults are software references, not an approved B engineering model. B VCU odometry uses coarse speed feedback and commanded steering; account for that limitation in localisation.

Source and edition

Prepared and reorganised by RJ Tech from the already-public Luchs ROS 2 v1.1.0 package by TerraAgent Robotics. Source licence: Apache-2.0. Source text and examples were shortened, explained and scoped for this topic. Original archives and protocol files remain unchanged.

Source file: docs/integration.md

Source file: README.md

Source file: luchs_description/README.md

Original public software archive | Source licence